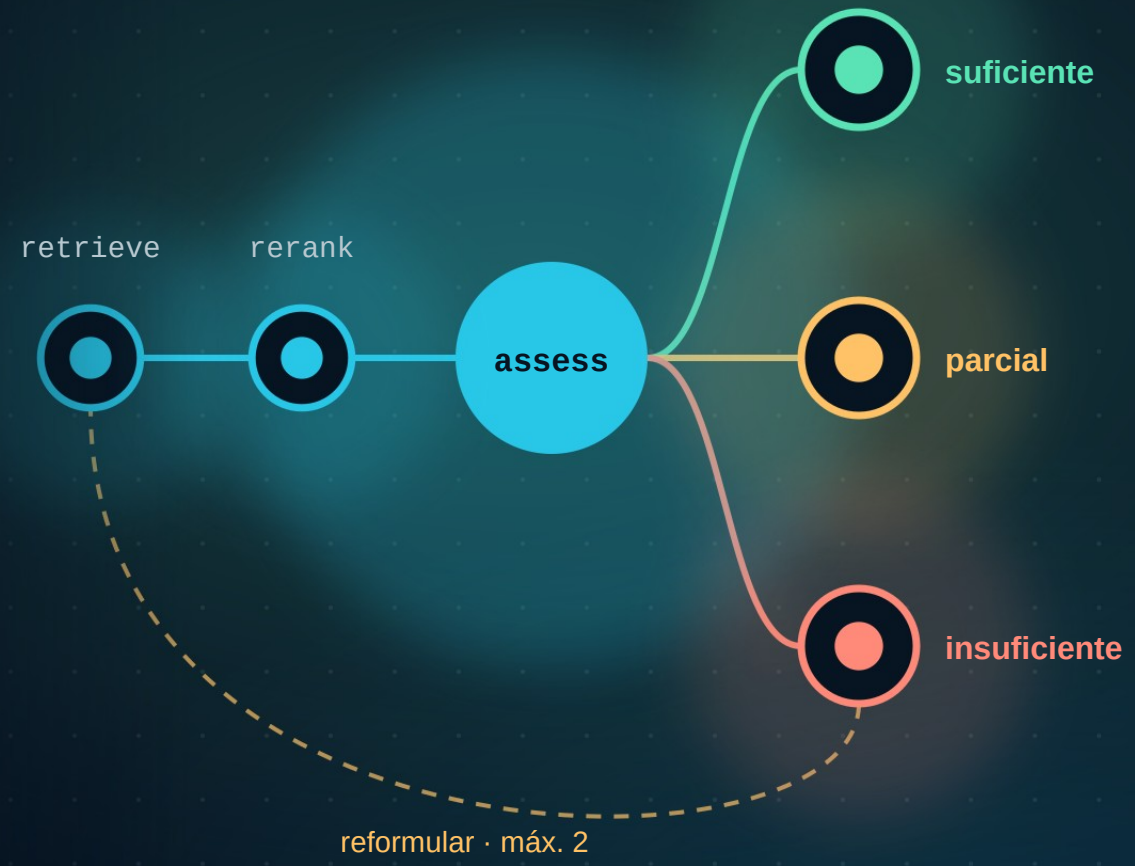
thePower

S11 · AI ENGINEER

CRAG

RAG avanzado: cuando una búsqueda no basta

Reordenar, evaluar la evidencia y corregir la recuperación, con límites y medido frente al RAG básico.



José Antonio Nuevo

Martes 27 de octubre de 2026 · 18:30–20:00

20 min conceptos · 55 min demo · 15 min evaluación y dudas

ANTES DE EMPEZAR · 1 MINUTO

Evalúa la clase anterior: S10 · RAG

Encuesta AI Engineer

Abre la cámara del móvil y escanea

1

Escanea el QR

La encuesta valora la sesión S10 del jueves 22 de octubre, no la de hoy.

2

Responde con sinceridad

Ritmo, profundidad, demo y ejercicio. Son menos de dos minutos.

3

Es muy importante

Con vuestras respuestas ajustamos las próximas clases. Sin ellas no sabemos qué mejorar.

Mientras respondéis: pensad en una pregunta que el RAG de S10 no resolvió bien a la primera.

MAPA DE LA SESIÓN · 90 MIN

Lo que pide el temario, en tres bloques



1 · RAG AVANZADO Y CRAG

Conceptos

- Límites del RAG básico
- Técnicas de autocorrección y reordenamiento
- El grafo CRAG y sus tres rutas
- Límites y salidas del turno

2 · CORRECTIVE RAG EN VIVO

Demo

- Seis guiones en el asistente del curso
- El laboratorio pgvector en terminal
- Leer la traza: consultas, puntuaciones, descartes

3 · EVALUACIÓN Y MÉTRICAS

Medir

- Métricas específicas de sistemas RAG
- Comparación justa y resultados
- Ejercicio: un CRAG completo desde cero

Temario thePower · RAG avanzado · Corrective RAG · Evaluación y métricas específicas de sistemas RAG · Ejercicio: construir un CRAG completo desde cero.

EL LÍMITE DEL RAG BÁSICO

Una respuesta puede sonar bien y estar incompleta



El fallo no está en el generador: está en aceptar evidencia parcial como si bastara.

Dónde se puede mejorar un RAG básico

Objetivo del temario: superar sus limitaciones con autocorrección y reordenamiento de resultados.

Antes de buscar

- **Reformular la consulta**
- Descomponer preguntas compuestas
- Multi-query o HyDE

Al buscar

- **Top-k amplio: hasta 10 candidatos**
- Filtros por grupo y versión
- Búsqueda híbrida: léxica + vectorial

Después de buscar

- **Reordenar con un reranker**
- **Evaluar si la evidencia basta**
- Self-RAG: el modelo critica su recuperación

Al generar

- **Contexto sin duplicados**
- **Rechazar si no consta**
- Citas verificables

Lo que usa el CRAG de S11: reformulación, top-10, reordenamiento, evaluación de la evidencia y rechazo, en un mismo grafo.

CORRECTIVE RAG

CRAG mete un juicio entre recuperar y generar

Recuperar

`retrieve`

Trae hasta 10 candidatos desde el índice vectorial.

→

Reordenar

`rerank`

Puntúa cada fragmento en [0, 1] frente a la consulta.

→

Juzgar

`assess`

¿La evidencia útil cubre toda la pregunta?

→

Corregir

`retain · discard`

Conserva o descarta, reformula y vuelve a buscar. Si basta, responde.

✓ **Qué es** Una política de recuperación observable: cada decisión e intento queda en la traza.

✗ **Qué no es** No es GraphRAG, no entrena el modelo y no garantiza mejorar cada respuesta.

Inspirado en Yan, Gu, Zhu y Ling (2024), Corrective Retrieval Augmented Generation, arXiv:2401.15884. Es una adaptación docente, no una reproducción del artículo

PUNTO DE PARTIDA

Mismo asistente, otra política de recuperación

Pieza	RAG básico · S10	CRAG · S11
Búsquedas	1 búsqueda (top-4)	**Hasta 3 (top-10 y 2 reformulaciones)**
Control	Un umbral de similitud	**rerank en [0, 1] + assess de cobertura**
Contexto	Lo recuperado una vez	**Conserva, une sin duplicados o descarta**
Decisión	Responder o rechazar	**Suficiente, parcial o insuficiente; responde o rechaza**
Coste	Una recuperación	**Más llamadas y más latencia**

IGUAL EN LOS DOS

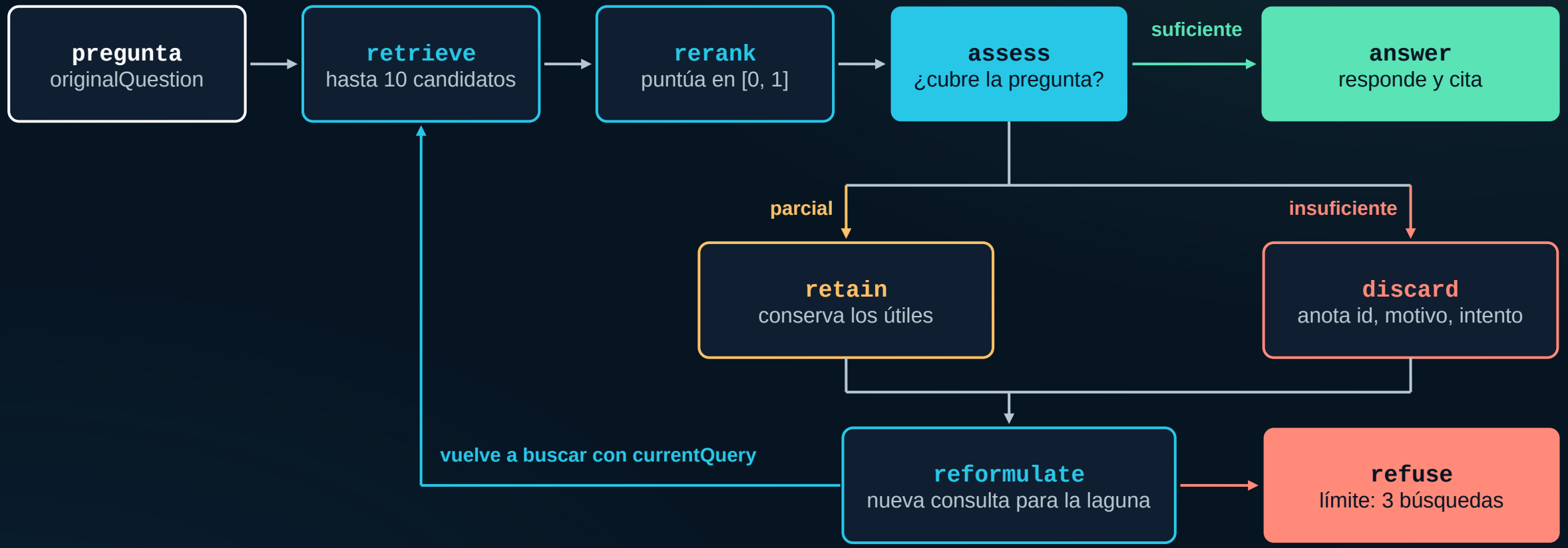
- Corpus y fragmentos
- Índice y modelo de embeddings
- Modelo generativo
- Instrucciones base
- Conversaciones limpias

Solo cambia la política de corrección.

El RAG básico sigue en el mismo proyecto: es la línea base contra la que medimos CRAG.

ARQUITECTURA

El grafo de LangGraph, nodo a nodo



Cada nodo devuelve datos estructurados; las aristas condicionales deciden la ruta. refuse termina sin llamar al generador.

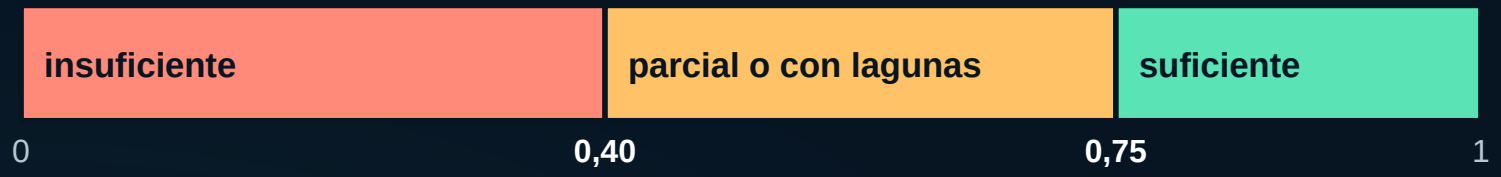
REORDENAR Y EVALUAR

Relevancia de un fragmento ≠ cobertura de la pregunta

`rerank` ¿Este fragmento es relevante para la consulta?



`assess` ¿La evidencia útil cubre toda la pregunta?



EJEMPLO DE LA DEMO

Fragmento de S5
`rerank = 0,80 → útil`

Pero falta la observabilidad
`cobertura = 0,50 → parcial`

Un fragmento muy relevante puede cubrir solo media pregunta.

Umbrales calibrados con ejemplos de validación, nunca con el test. Una puntuación no es una probabilidad de acierto.

LAS RUTAS CORRECTIVAS

Parcial conserva; insuficiente descarta

PARCIAL *guion «Evidencia parcial y segunda búsqueda»*

«¿Cómo modela LangGraph el estado de un agente y cómo se observa esa ejecución?»

1. `retrieve 1 · assess`
   S5 (0,80) es útil, pero la cobertura es 0,50
2. `retain`
   Conserva S5 con su procedencia
3. `reformulate`
   «observabilidad Langfuse y LangSmith de un agente»
4. `retrieve 2 · answer`
   S6 (0,86) + S5 sin duplicar: responde citando ambos

INSUFICIENTE *guion «Primera evidencia descartada»*

«¿Qué cubre la sesión de observabilidad del programa?»

1. `retrieve 1 · assess`
   S16 · Cloud AWS (0,20 < 0,50): nada útil
2. `discard`
   Anota id, motivo e intento 1
3. `reformulate`
   Nueva consulta desde la pregunta original
4. `retrieve 2 · answer`
   S6 (0,88): responde. S16 nunca entra en el prompt

ESTADO DEL TURNO

La pregunta original nunca se sobrescribe

`originalQuestion`

Lo que preguntó el alumno. Llega intacta a assess y a answer.

`currentQuery`

La consulta de búsqueda. Cambia en cada reformulación.

`queries`

Consultas ya usadas: una consulta repetida corta el bucle.

Cada turno nace vacío: contadores a 0 y sin checkpointer que arrastre estado de otra pregunta.

lab/src/crag-graph.ts · extracto

```
Annotation.Root({
  originalQuestion: Annotation<string>,
  currentQuery: Annotation<string>,
  queries: Annotation<string[]>,
  retrievalCount: Annotation<number>,      // máx. 3
  reformulationCount: Annotation<number>, // máx. 2
  retained: Annotation<LabVectorHit[]>,
  discarded: Annotation<LabDiscardedRecord[]>,
  terminal: Annotation<"pending" | "answer" | "refuse">,
})
```

LÍMITES Y SALIDAS

Los límites viven en el código, no en el prompt

3
búsquedas
incluida la primera; un fallo también cuenta

2
reformulaciones
como máximo, sin reintentos ocultos

20 s
plazo del turno
en el asistente; cada llamada usa lo que queda

0
llamadas al LLM
para generar cuando el turno termina en rechazo

CUATRO SALIDAS QUE NO SE MEZCLAN

answer
Evidencia suficiente y citas que existen en la evidencia permitida.

refuse
Se agotan los intentos o la consulta se repite. Dice «no consta».

clarify
Pregunta de calendario sin grupo: pide aclaración antes de buscar.

error
Falla una dependencia o se acaba el plazo. No es un rechazo correcto.

Separar rechazo y error evita que un fallo de infraestructura parezca una decisión prudente.

DEMO · 55 MIN

Seis guiones, seis caminos del grafo

Suficiente a la primera	`answer en la búsqueda 1`
Evidencia parcial y segunda búsqueda	`retain → reformulate → answer`
Primera evidencia descartada	`discard → reformulate → answer`
Dato ausente y rechazo	`3 búsquedas → refuse sin generar`
Calendario sin cohorte	`clarify con 0 búsquedas`
Fallo controlado del evaluador	`error, sin reintentos`



Asistente del curso

`rag-thepower.josenuevo.dev`

Etapa CRAG · grupo M/J

En la traza: consultas, puntuaciones, conservados y descartados



Laboratorio del ejercicio

`bun run lab:demo`

PostgreSQL + pgvector · 10 documentos AetherCloud

Caso compuesto mTLS + SEV-1 y caso de rechazo

Los guiones enseñan el recorrido; no miden la calidad del modelo. Respaldo local: bun run dev → http://localhost:5173/

EVALUACIÓN Y MÉTRICAS

Qué medir en un sistema RAG

RECUPERACIÓN

¿Llega la evidencia?

- Recall@k: ¿está el fragmento esperado en el top-k?
- MRR y nDCG: ¿en qué posición aparece?
- Precisión del contexto: ¿cuánto ruido entra?

GENERACIÓN

¿La respuesta se apoya en ella?

- **Acierto frente a la referencia (X/14)**
- **Fuentes citadas revisadas a mano**
- Fidelidad: nada fuera del contexto

ABSTENCIÓN

¿Sabe decir «no consta»?

- **Rechazos correctos (Y/6)**
- Falsos rechazos en preguntas con respuesta
- Datos inventados sin evidencia

OPERACIÓN

¿Cuánto cuesta corregir?

- **Búsquedas y reformulaciones medias**
- **Latencia mediana y media**
- Tokens y coste conocido

- Pedido en el ejercicio S11. Ragas automatiza fidelidad y precisión del contexto con un modelo juez; aquí no se exige.

1 · Desarrollo
casos visibles

→

2 · Validación
calibrar umbrales y prompts

→

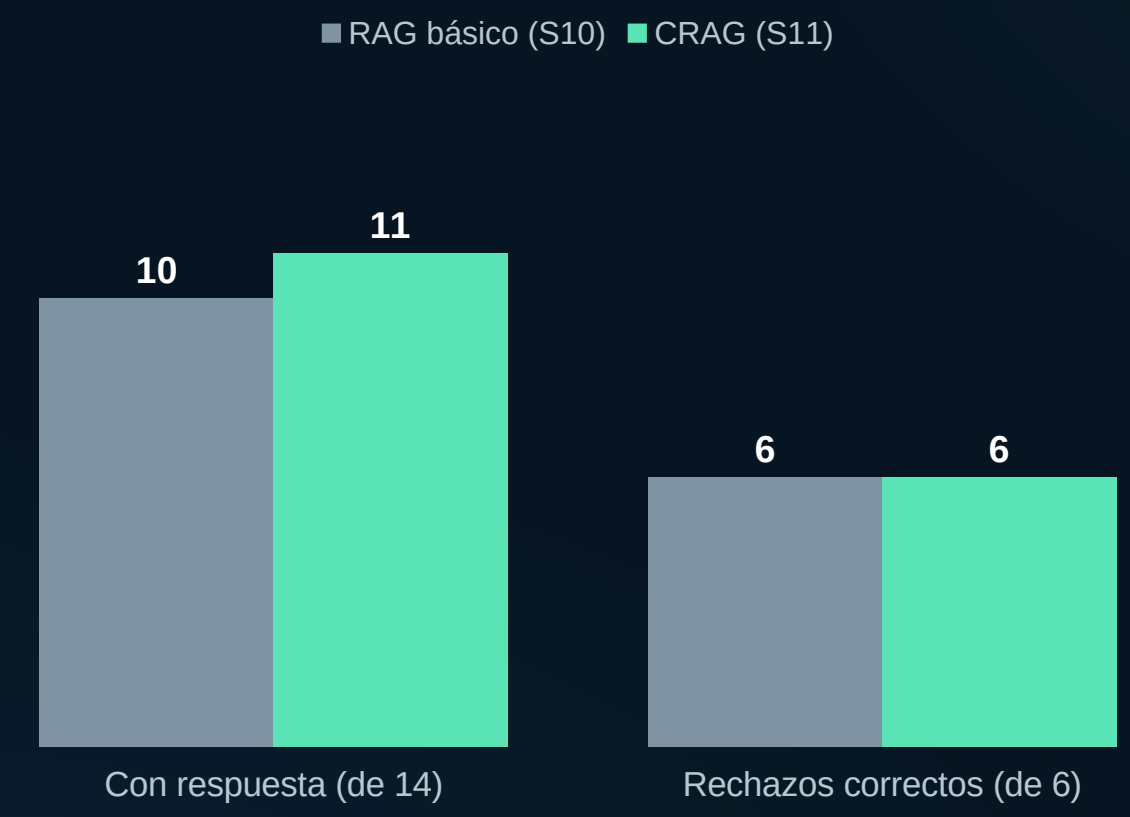
3 · Congelar
configuración y versiones

→

4 · Test
20 nuevas: 14 + 6

RESULTADOS DEL LABORATORIO · 20 PREGUNTAS

CRAG acierta una más y casi duplica las búsquedas



16 → 17

Aciertos sobre 20. CRAG gana las preguntas que unen dos documentos (casos 5, 10 y 14).

1,00 → 1,80

Búsquedas medias. Cada pregunta sin respuesta agota 3 búsquedas antes de rechazar.

2 fallos nuevos

Casos 3 y 8. Cobertura parcial tras 2 reformulaciones: CRAG rechaza por prudencia.

lab/ · 10 documentos AetherCloud, vector(768) + HNSW, Qwen3-Reranker-0.6B (lab/eval/resultados-laboratorio.md). Lote histórico del asistente (3 oct): 17/20 → 18/20 y latencia mediana 1,19 s → 4,30 s. Muestra pequeña: no es una tasa general.

EJERCICIO S11 · 6 A 8 HORAS

Construir un CRAG completo desde cero

- **Base reconstruible**
  8+ documentos ficticios e índice vacío: PostgreSQL + pgvector, vector(768) y HNSW.
- **Línea base**
  El RAG básico de S10 sigue operativo en el mismo proyecto.
- **Grafo separado**
  retrieve (10) → rerank [0, 1] → assess → retain / discard → reformulate → answer / refuse.
- **Trazas**
  Las 3 rutas, un rechazo y la prueba del límite: 3 búsquedas y 2 reformulaciones.
- **Comparación ejecutada**
  20 preguntas nuevas: X/14, Y/6, búsquedas medias, latencia y revisión de fuentes.

Turno de dudas.

NO SE EXIGE

- Ragas
- Interfaz gráfica
- Búsqueda web (opcional)
- Qwen3-Reranker-0.6B (opcional)
- Que CRAG gane siempre

Material de apoyo

`lab/` y `laboratorio-apoyo.md`